

MicroserviceClewareSwitch

v. 0.1.2

TODO

02.02.2023

Contents

1	Introduction	1
2	Description	2
2.1	ToDo	2
2.1.1	ToDo ToDo	2
3	ClewareAccessHelper.py	3
3.1	Class: ClewareAccessHelper	3
3.1.1	Method: init_cleware	4
3.1.2	Method: open_cleware	4
3.1.3	Method: close_cleware	4
3.1.4	Method: get_handle	4
3.1.5	Method: set_value	4
3.1.6	Method: get_value	4
3.1.7	Method: set_switch	4
3.1.8	Method: set_switch_by_port_name	4
3.1.9	Method: get_switch	4
3.1.10	Method: get_version	4
3.1.11	Method: get_usb_type	4
3.1.12	Method: get_serial_number	4
3.1.13	Method: valid_ser_number	4
3.1.14	Method: get_hw_version	4
3.1.15	Method: is_ampel	4
3.1.16	Method: iox	4
3.1.17	Method: get_all_devices_state	4
4	ClewareAccessHelperAbs.py	5
4.1	Class: ClewareAccessHelperAbs	5
4.1.1	Method: open_cleware	5
4.1.2	Method: close_cleware	5
4.1.3	Method: set_value	5
4.1.4	Method: get_value	5
4.1.5	Method: set_switch	5
4.1.6	Method: get_switch	5
4.1.7	Method: get_handle	5
4.1.8	Method: get_version	5
4.1.9	Method: get_usb_type	5
4.1.10	Method: get_usb_type	5

4.1.11	Method: <code>get_serial_number</code>	5
4.1.12	Method: <code>valid_ser_number</code>	5
4.1.13	Method: <code>get_hw_version</code>	5
4.1.14	Method: <code>is_ampel</code>	5
4.1.15	Method: <code>iox</code>	5
5	ClewareAccessHelperLinux.py	6
5.1	Class: <code>ClewareAccessHelperLinux</code>	6
5.1.1	Method: <code>init_cleware</code>	7
5.1.2	Method: <code>open_cleware</code>	7
5.1.3	Method: <code>close_cleware</code>	7
5.1.4	Method: <code>get_handle</code>	7
5.1.5	Method: <code>set_value</code>	7
5.1.6	Method: <code>get_value</code>	7
5.1.7	Method: <code>set_switch</code>	7
5.1.8	Method: <code>get_switch</code>	7
5.1.9	Method: <code>get_version</code>	7
5.1.10	Method: <code>get_usb_type</code>	7
5.1.11	Method: <code>get_serial_number</code>	7
5.1.12	Method: <code>valid_ser_number</code>	7
5.1.13	Method: <code>get_hw_version</code>	7
5.1.14	Method: <code>is_ampel</code>	7
5.1.15	Method: <code>iox</code>	7
5.1.16	Method: <code>get_all_sw_state</code>	7
6	ClewareAccessHelperWindows.py	8
6.1	Class: <code>ClewareAccessHelperWindows</code>	8
6.1.1	Method: <code>init_cleware</code>	8
6.1.2	Method: <code>open_cleware</code>	8
6.1.3	Method: <code>close_cleware</code>	8
6.1.4	Method: <code>get_handle</code>	8
6.1.5	Method: <code>set_value</code>	8
6.1.6	Method: <code>get_value</code>	8
6.1.7	Method: <code>set_switch</code>	8
6.1.8	Method: <code>get_switch</code>	8
6.1.9	Method: <code>get_version</code>	8
6.1.10	Method: <code>get_usb_type</code>	8
6.1.11	Method: <code>get_serial_number</code>	8
6.1.12	Method: <code>valid_ser_number</code>	8
6.1.13	Method: <code>get_hw_version</code>	8
6.1.14	Method: <code>is_ampel</code>	8
6.1.15	Method: <code>iox</code>	8
7	ServiceBase.py	9
7.1	Class: <code>ResultType</code>	9
7.2	Class: <code>ResponseMessage</code>	9
7.2.1	Method: <code>get_json</code>	9

7.2.2	Method: <code>get_data</code>	9
7.3	Class: <code>ServiceBase</code>	9
7.3.1	Method: <code>parse_arguments</code>	10
7.3.2	Method: <code>parse_spec_arguments</code>	10
7.3.3	Method: <code>close</code>	10
7.3.4	Method: <code>create_request_data</code>	10
7.3.5	Method: <code>connect_broker</code>	11
7.3.6	Method: <code>serve</code>	11
7.3.7	Method: <code>register_service</code>	11
7.3.8	Method: <code>unregister_service</code>	11
7.3.9	Method: <code>request_service</code>	11
7.3.10	Method: <code>get_svc_api_methods_dict</code>	12
7.3.11	Method: <code>get_svc_api_methods_info_dict</code>	12
7.3.12	Method: <code>parse_docstring</code>	12
7.3.13	Method: <code>svc_api_get_version</code>	12
7.3.14	Method: <code>svc_api_get_gui_files</code>	12
7.3.15	Method: <code>svc_api_get_sample</code>	12
7.3.16	Method: <code>is_specific_request</code>	12
7.3.17	Method: <code>on_specific_request</code>	13
7.3.18	Method: <code>on_request</code>	13
8	ServiceCleware.py	14
8.1	Function: <code>signal_handler</code>	14
8.2	Class: <code>ServiceCleware</code>	14
8.2.1	Method: <code>svc_api_get_all_devices_state</code>	14
8.2.2	Method: <code>svc_api_set_switch</code>	15
8.2.3	Method: <code>notify_updates</code>	15
9	ServiceLogger.py	16
9.1	Class: <code>ServiceLogger</code>	16
9.1.1	Method: <code>get_config_file</code>	16
9.1.2	Method: <code>log</code>	16
9.1.3	Method: <code>log_info</code>	16
9.1.4	Method: <code>log_debug</code>	16
9.1.5	Method: <code>log_warning</code>	16
9.1.6	Method: <code>log_error</code>	16
9.1.7	Method: <code>log_critical</code>	16
10	Utils.py	17
10.1	Class: <code>Singleton</code>	17
10.2	Class: <code>Utils</code>	17
10.2.1	Method: <code>get_all_sub_classes</code>	17
10.2.2	Method: <code>caller_name</code>	17
10.2.3	Method: <code>load_library</code>	18
10.2.4	Method: <code>is_ascii_or_unicode</code>	18
10.2.5	Method: <code>get_registry_parameters</code>	18
10.2.6	Method: <code>create_registry_parameters</code>	18

10.3	Class: Job	18
10.3.1	Method: stop	18
10.3.2	Method: run	18
10.4	Class: ResultType	18
11	Appendix	19
12	History	20

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceClewareSwitch**.

The sources of **MicroserviceClewareSwitch** are available in [GitHub](#).

Informations about how to install the **MicroserviceClewareSwitch** can be found in the [README](#).

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

ClewareAccessHelper.py

3.1 Class: ClewareAccessHelper

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelper import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-load-config -----

Load the user config port name for USB access. Returns: None microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-get-config -----

Get Cleware ports' config. Returns: Dictionary of ports' setting. microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-set-config -----

Save Cleware ports' config to file. Args: config.json: data to be saved.

Returns: res: saved result.

- 3.1.1 Method: `init_cleware`
- 3.1.2 Method: `open_cleware`
- 3.1.3 Method: `close_cleware`
- 3.1.4 Method: `get_handle`
- 3.1.5 Method: `set_value`
- 3.1.6 Method: `get_value`
- 3.1.7 Method: `set_switch`
- 3.1.8 Method: `set_switch_by_port_name`
- 3.1.9 Method: `get_switch`
- 3.1.10 Method: `get_version`
- 3.1.11 Method: `get_usb_type`
- 3.1.12 Method: `get_serial_number`
- 3.1.13 Method: `valid_ser_number`
- 3.1.14 Method: `get_hw_version`
- 3.1.15 Method: `is_ampel`
- 3.1.16 Method: `iox`
- 3.1.17 Method: `get_all_devices_state`

Chapter 4

ClewareAccessHelperAbs.py

4.1 Class: ClewareAccessHelperAbs

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelperAbs import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microserviceclewareswitch-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware -----

4.1.1 Method: open_cleware

4.1.2 Method: close_cleware

4.1.3 Method: set_value

4.1.4 Method: get_value

4.1.5 Method: set_switch

4.1.6 Method: get_switch

4.1.7 Method: get_handle

4.1.8 Method: get_version

4.1.9 Method: get_usb_type

4.1.10 Method: get_usb_type

4.1.11 Method: get_serial_number

4.1.12 Method: valid_ser_number

4.1.13 Method: get_hw_version

4.1.14 Method: is_ampel

4.1.15 Method: iox

Chapter 5

ClewareAccessHelperLinux.py

5.1 Class: ClewareAccessHelperLinux

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelperLinux import  
↪ ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

- 5.1.1 Method: `init_cleware`
- 5.1.2 Method: `open_cleware`
- 5.1.3 Method: `close_cleware`
- 5.1.4 Method: `get_handle`
- 5.1.5 Method: `set_value`
- 5.1.6 Method: `get_value`
- 5.1.7 Method: `set_switch`
- 5.1.8 Method: `get_switch`
- 5.1.9 Method: `get_version`
- 5.1.10 Method: `get_usb_type`
- 5.1.11 Method: `get_serial_number`
- 5.1.12 Method: `valid_ser_number`
- 5.1.13 Method: `get_hw_version`
- 5.1.14 Method: `is_ampel`
- 5.1.15 Method: `iox`
- 5.1.16 Method: `get_all_sw_state`

Chapter 6

ClewareAccessHelperWindows.py

6.1 Class: ClewareAccessHelperWindows

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelperWindows import  
↳ ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

6.1.1 Method: init_cleware

6.1.2 Method: open_cleware

6.1.3 Method: close_cleware

6.1.4 Method: get_handle

6.1.5 Method: set_value

6.1.6 Method: get_value

6.1.7 Method: set_switch

6.1.8 Method: get_switch

6.1.9 Method: get_version

6.1.10 Method: get_usb_type

6.1.11 Method: get_serial_number

6.1.12 Method: valid_ser_number

6.1.13 Method: get_hw_version

6.1.14 Method: is_ampel

6.1.15 Method: iox

Chapter 7

ServiceBase.py

7.1 Class: ResultType

Imported by:

```
from MicroserviceClewareSwitch.ServiceBase import ResultType
```

Result Types.

7.2 Class: ResponseMessage

Imported by:

```
from MicroserviceClewareSwitch.ServiceBase import ResponseMessage
```

Response message class

7.2.1 Method: get_json

Convert the response message to JSON format.

Returns:

/ Type: str /

The response message in JSON format.

7.2.2 Method: get_data

Retrieve the result data as a string.

Returns:

/ Type: str /

The result data as a string.

7.3 Class: ServiceBase

Imported by:

```
from MicroserviceClewareSwitch.ServiceBase import ServiceBase
```

7.3.1 Method: `parse_arguments`

Parse basic service arguments from the command line.

Arguments:

- `cmd_args`
/ *Condition*: required / *Type*: list /
Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed arguments.

7.3.2 Method: `parse_spec_arguments`

Parse specific arguments for each customized service.

This method should be overridden by subclasses to handle service-specific arguments.

Arguments:

- `cmd_args`
/ *Condition*: required / *Type*: list /
Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed specific arguments.

7.3.3 Method: `close`

Close the service connection.

Returns:

(no returns)

7.3.4 Method: `create_request_data`

Create request data for a given method name and arguments.

Arguments:

- `method_name`
/ *Condition*: required / *Type*: str /
The name of the method for which the request is being created.
- `args`
/ *Condition*: required / *Type*: list /
The list of arguments for the method.

Returns:

/ *Type*: dict /
A dictionary containing the method name and arguments.

7.3.5 Method: connect_broker

Establish a connection to the broker.

Arguments:

- `**kwargs`
/ *Condition*: optional / *Type*: dict /
Additional keyword arguments for broker connection parameters.

Returns:

(no returns)

7.3.6 Method: serve

Call to start service serving.

Returns:

(no returns)

7.3.7 Method: register_service

Register a service to the ServiceRegistry.

Returns:

(no returns)

7.3.8 Method: unregister_service

Unregister a service from the ServiceRegistry.

Returns:

(no returns)

7.3.9 Method: request_service

Send a service request to a specific exchange with a given routing key.

Arguments:

- `request_data`
/ *Condition*: required / *Type*: dict /
The data for the service request.
- `exchange_name`
/ *Condition*: required / *Type*: str /
The name of the exchange to send the request to.
- `routing_key`
/ *Condition*: required / *Type*: str /
The routing key for the request.

Returns:

(no returns)

7.3.10 Method: `get_svc_api_methods_dict`

Retrieve all service API methods provided by the service (methods starting with the prefix 'svc_api').

Returns:

/ Type: dict /

A dictionary containing the names and references of all service API methods.

7.3.11 Method: `get_svc_api_methods_info_dict`

Retrieve information for all service APIs from the docstrings of the methods.

Arguments:

- `methods_dict`

/ Condition: required / Type: dict /

A dictionary containing the names and references of all service API methods.

Returns:

/ Type: dict /

A dictionary containing the information of all service APIs extracted from their docstrings.

7.3.12 Method: `parse_docstring`

Parse function's docstring to get arguments and return information.

Arguments:

- `docstring`

/ Condition: required / Type: str /

Function's docstring.

Returns:

/ Type: str /

Payloads of the waiting signal if received.

7.3.13 Method: `svc_api_get_version`

Get the service version.

Returns:

/ Type: str /

Version of the service.

7.3.14 Method: `svc_api_get_gui_files`

7.3.15 Method: `svc_api_get_sample`

7.3.16 Method: `is_specific_request`

Check if the request is a specific request.

Arguments:

- request
/ *Condition*: required / *Type*: object /
The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

7.3.17 Method: on_specific_request

Handle the event when a specific request is received.

Arguments:

- ch
/ *Condition*: required / *Type*: pika.channel.Channel /
The channel object from the pika library.
- method
/ *Condition*: required / *Type*: pika.spec.Basic.Deliver /
The method object containing delivery information from the pika library.
- props
/ *Condition*: required / *Type*: pika.spec.BasicProperties /
The properties of the message from the pika library.
- body
/ *Condition*: required / *Type*: dict /
The body of the message as a dictionary.

Returns:

(no returns)

7.3.18 Method: on_request

Handle an incoming request.

Arguments:

- ch
/ *Condition*: required / *Type*: pika.channel.Channel /
The channel object from the pika library.
- method
/ *Condition*: required / *Type*: pika.spec.Basic.Deliver /
The method object containing delivery information from the pika library.
- props
/ *Condition*: required / *Type*: pika.spec.BasicProperties /
The properties of the message from the pika library.
- body
/ *Condition*: required / *Type*: bytes /
The body of the message as bytes.

Returns:

(no returns)

Chapter 8

ServiceCleware.py

8.1 Function: `signal_handler`

Handle signals from the operating system.

Arguments:

- `sig`
/ *Condition*: required / *Type*: int /
The signal number received from the OS.
- `frame`
/ *Condition*: required / *Type*: frame object /
The current stack frame.
- `obj`
/ *Condition*: required / *Type*: object /
The object that is handling the signal.

Returns:

(no returns)

8.2 Class: `ServiceCleware`

Imported by:

```
from MicroserviceClewareSwitch.ServiceCleware import ServiceCleware
```

Service for controlling Cleware devices.

This class extends `ServiceBase` to provide functionalities specific to managing and controlling Cleware devices.

8.2.1 Method: `svc_api_get_all_devices_state`

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

8.2.2 Method: `svc_api_set_switch`

Set state for a Cleware device's switch.

Arguments:

- `device_no`
/ *Condition*: required / *Type*: str /
Cleware device's number.
- `switch_id`
/ *Condition*: required / *Type*: str /
Switch number to turn on or off.
- `state`
/ *Condition*: required / *Type*: str /
State of switch to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

8.2.3 Method: `notify_updates`

Notify updates to the realtime update channel for Cleware devices.

Returns:

(no returns)

Chapter 9

ServiceLogger.py

9.1 Class: ServiceLogger

Imported by:

```
from MicroserviceClewareSwitch.ServiceLogger import ServiceLogger
```

Logger class. microserviceclewareswitch-servicelogger-servicelogger-set-logger -----

9.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

9.1.2 Method: log

9.1.3 Method: log_info

9.1.4 Method: log_debug

9.1.5 Method: log_warning

9.1.6 Method: log_error

9.1.7 Method: log_critical

Chapter 10

Utils.py

10.1 Class: Singleton

Imported by:

```
from MicroserviceClewareSwitch.Utils import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

10.2 Class: Utils

Imported by:

```
from MicroserviceClewareSwitch.Utils import Utils
```

Class to implement utilities for supporting development. microserviceclewareswitch-utils-utils-get-all-descendant-classes -----

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

10.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

10.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

10.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

10.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

10.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

10.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

10.3 Class: Job

Imported by:

```
from MicroserviceClewareSwitch.Utls import Job
```

10.3.1 Method: stop

10.3.2 Method: run

10.4 Class: ResultType

Imported by:

```
from MicroserviceClewareSwitch.Utls import ResultType
```

Result Types. microserviceclewareswitch-utils-responsemessage =====

Imported by:

```
from MicroserviceClewareSwitch.Utls import ResponseMessage
```

Response message class microserviceclewareswitch-utils-responsemessage-get-json -----

Convert response message to json Returns: Response message in json format microserviceclewareswitch-utils-responsemessage-get-data -----

Get string data result Returns: String result microserviceclewareswitch-utils-responsemessage-create-from-string -----

Chapter 11

Appendix

About this package:

Table 11.1: Package setup

Setup parameter	Value
Name	MicroserviceClewareSwitch
Version	0.1.2
Date	02.02.2023
Description	TODO
Package URL	python-microservice-cleware-switch
Author	TODO
Email	TODO
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 12

History

0.1.0	02/2023
<i>Initial version</i>	